

PROGRAMACIÓN APLICADA 2026-2

GUÍA EXPLICATIVA DEL SCRIPT POSTGRESQL

ParkFlow 360 · Creación completa de tablas V1

Cómo leer, entender, ejecutar y defender el DDL completo en DataGrip

Motor	PostgreSQL
Herramienta de ejecución	DataGrip
Script base	ParkFlow360_V1_Creacion_Completa_PostgreSQL.sql
Alcance	Schema + 16 tablas + constraints + índices + verificación
Propósito	Comprender qué hace cada línea importante y por qué el modelo protege la integridad de ParkFlow 360

Regla de esta guía

No memorices el script como texto. Debes poder explicar qué problema resuelve cada PK, FK, UNIQUE, CHECK, NOT NULL e índice, y qué error permitiría la base si esa protección no existiera.

1. Qué vas a aprender con este script

- Cómo PostgreSQL transforma un modelo lógico en tablas físicas.
- Cómo una PRIMARY KEY identifica una fila y cómo IDENTITY genera IDs automáticamente.
- Cómo una FOREIGN KEY materializa relaciones entre tablas.
- Cómo UNIQUE protege reglas de no repetición, tanto globales como compuestas.
- Cómo NOT NULL obliga a que un dato exista.
- Cómo CHECK impide estados, rangos o combinaciones inválidas dentro de una fila.
- Por qué el orden de creación de las tablas depende de las claves foráneas.
- Por qué algunas reglas no pueden resolverse sólo con constraints simples.
- Qué función cumplen los índices y por qué una FK no queda indexada automáticamente en PostgreSQL.
- Cómo verificar que las 16 tablas fueron creadas correctamente.

Idea central

El script es la versión ejecutable del modelo de datos. El DER explica relaciones; el DDL obliga a PostgreSQL a respetarlas.

2. Cómo leer un CREATE TABLE

```
CREATE TABLE ejemplo (
    ejemplo_id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    codigo    VARCHAR(30) NOT NULL,
    padre_id  BIGINT NOT NULL,

    CONSTRAINT pk_ejemplo PRIMARY KEY (ejemplo_id),
    CONSTRAINT fk_ejemplo_padre FOREIGN KEY (padre_id) REFERENCES padre (padre_id),
    CONSTRAINT uq_ejemplo_codigo UNIQUE (codigo),
    CONSTRAINT ck_ejemplo_codigo CHECK (BTRIM(codigo) <> '')
);
```

CREATE TABLE	Ordena a PostgreSQL crear una estructura persistente para filas del mismo tipo.
BIGINT	Número entero grande; en ParkFlow se usa para identificadores técnicos.
GENERATED BY DEFAULT AS IDENTITY	PostgreSQL genera automáticamente el siguiente ID cuando no lo enviamos.
VARCHAR(n)	Texto con longitud máxima definida.
NOT NULL	La columna es obligatoria.
PRIMARY KEY	Identificador único y no nulo de la fila.
FOREIGN KEY	Referencia obligatoria u opcional a una fila válida de otra tabla.
UNIQUE	Impide duplicar el valor o combinación indicada.
CHECK	Valida una condición dentro de la fila antes de aceptar el INSERT/UPDATE.
CONSTRAINT nombre	Da un nombre explícito a la regla para identificarla claramente cuando PostgreSQL reporta un error.

3. Antes de ejecutar: schema y search_path

```
CREATE SCHEMA IF NOT EXISTS parkflow;  
SET search_path TO parkflow, public;
```

- `CREATE SCHEMA IF NOT EXISTS parkflow` crea un espacio lógico llamado parkflow. No es otra base de datos: es un contenedor de objetos dentro de la base actual.
- `IF NOT EXISTS` evita que la sentencia falle si el schema ya existe.
- `SET search\_path TO parkflow, public` indica que las tablas sin prefijo se buscarán primero en parkflow.
- Después de esta línea, `CREATE TABLE usuario (...)` crea realmente `parkflow.usuario`.

Pregunta de defensa

¿Cuál es la diferencia entre una base de datos PostgreSQL y un schema? Respuesta esperada: la base contiene schemas; el schema organiza tablas y otros objetos dentro de esa base.

4. Tabla USUARIO

Propósito: Cuenta capaz de autenticarse y ejecutar acciones.

```
CREATE TABLE usuario (
    usuario_id      BIGINT GENERATED BY DEFAULT AS IDENTITY,
    username        VARCHAR(60) NOT NULL,
    email           VARCHAR(160) NOT NULL,
    password_hash   VARCHAR(255) NOT NULL,
    nombre_completo VARCHAR(160) NOT NULL,
    activo          BOOLEAN      NOT NULL,
    created_at      TIMESTAMPTZ  NOT NULL,
    updated_at      TIMESTAMPTZ  NOT NULL,

    CONSTRAINT pk_usuario PRIMARY KEY (usuario_id),
    CONSTRAINT uq_usuario_username UNIQUE (username),
    CONSTRAINT uq_usuario_email UNIQUE (email)
);

COMMENT ON TABLE usuario IS 'Cuenta capaz de autenticarse y ejecutar acciones dentro de ParkFlow 360.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
usuario_id	BIGINT IDENTITY	NO	PK	ID técnico de la cuenta.	Permite referenciar al usuario desde otras tablas sin usar username/email como PK.
username	VARCHAR(60)	NO	UNIQUE	Nombre de inicio de sesión.	No se permiten dos cuentas con el mismo username.
email	VARCHAR(160)	NO	UNIQUE	Correo de la cuenta.	Evita cuentas duplicadas por email.
password_hash	VARCHAR(255)	NO	-	Hash de contraseña.	No debe almacenarse la contraseña en texto plano.
nombre_completo	VARCHAR(160)	NO	-	Nombre visible del usuario.	Separa identidad visible de username.
activo	BOOLEAN	NO	-	Habilita o deshabilita la cuenta.	Permite inactivar sin borrar historial.
created_at	TIMESTAMPTZ	NO	-	Momento de creación.	Trazabilidad temporal.
updated_at	TIMESTAMPTZ	NO	-	Última actualización.	Permite saber cuándo cambió la cuenta.

Constraints y relaciones que debes entender

- pk_usuario garantiza identidad única.
- uq_usuario_username impide dos usernames iguales.
- uq_usuario_email impide dos emails iguales.
- No existen FK porque USUARIO es una tabla raíz en esta V1.

¿Qué error evita?

Ejemplo de protección

Impide crear dos cuentas con el mismo username o email.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de usuario y por qué no usamos un dato visible del negocio como PK?
- ¿Por qué las columnas marcadas NOT NULL son obligatorias?
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Por qué esta tabla puede crearse antes que las tablas que la referencian?

5. Tabla ROL

Propósito: Catálogo de funciones/permisos asignables.

```
CREATE TABLE rol (
    rol_id          BIGINT GENERATED BY DEFAULT AS IDENTITY,
    codigo          VARCHAR(40) NOT NULL,
    nombre          VARCHAR(100) NOT NULL,
    descripcion     TEXT,
    activo          BOOLEAN      NOT NULL,

    CONSTRAINT pk_rol PRIMARY KEY (rol_id),
    CONSTRAINT uq_rol_codigo UNIQUE (codigo)
);

COMMENT ON TABLE rol IS 'Funcion o conjunto de permisos asignable a usuarios.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
rol_id	BIGINT IDENTITY	NO	PK	ID del rol.	Referencia estable desde usuario_rol.
codigo	VARCHAR(40)	NO	UNIQUE	Código técnico/funcional del rol.	Evita dos roles con el mismo código.
nombre	VARCHAR(100)	NO	-	Nombre visible.	Describe el rol.
descripcion	TEXT	SÍ	-	Detalle opcional.	Puede no ser necesario en todos los roles.
activo	BOOLEAN	NO	-	Vigencia lógica.	Permite retirar un rol sin borrar histórico.

Constraints y relaciones que debes entender

- pk_rol identifica el rol.
- uq_rol_codigo protege el código funcional.
- ROL también es raíz antes de crear la tabla puente.

¿Qué error evita?

Ejemplo de protección

Impide duplicar códigos de rol.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de rol y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Por qué esta tabla puede crearse antes que las tablas que la referencian?

6. Tabla USUARIO_ROL

Propósito: Tabla puente que resuelve la relación N:M entre usuario y rol.

```
CREATE TABLE usuario_rol (
    usuario_rol_id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    usuario_id     BIGINT     NOT NULL,
    rol_id        BIGINT     NOT NULL,
    asignado_at   TIMESTAMPTZ NOT NULL,
    activo        BOOLEAN    NOT NULL,

    CONSTRAINT pk_usuario_rol PRIMARY KEY (usuario_rol_id),
    CONSTRAINT fk_usuario_rol_usuario
        FOREIGN KEY (usuario_id)
        REFERENCES usuario (usuario_id),
    CONSTRAINT fk_usuario_rol_rol
        FOREIGN KEY (rol_id)
        REFERENCES rol (rol_id),
    CONSTRAINT uq_usuario_rol_usuario_rol UNIQUE (usuario_id, rol_id)
);

COMMENT ON TABLE usuario_rol IS 'Tabla puente que resuelve la relacion N:M entre usuario y rol.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
usuario_rol_id	BIGINT IDENTITY	NO	PK	ID técnico de la asignación.	Facilita auditoría y futuras extensiones.
usuario_id	BIGINT	NO	FK -> usuario	Usuario asignado.	No puede apuntar a un usuario inexistente.
rol_id	BIGINT	NO	FK -> rol	Rol asignado.	No puede apuntar a un rol inexistente.
asignado_at	TIMESTAMPTZ	NO	-	Fecha/hora de asignación.	Trazabilidad.
activo	BOOLEAN	NO	-	Si la asignación sigue vigente.	Evita borrar para desasignar.

Constraints y relaciones que debes entender

- fk_usuario_rol_usuario obliga a que el usuario exista.
- fk_usuario_rol_rol obliga a que el rol exista.
- UNIQUE(usuario_id, rol_id) evita asignar dos veces el mismo rol al mismo usuario.
- Esta tabla transforma una N:M en dos relaciones 1:N.

¿Qué error evita?

Ejemplo de protección

Impide enlazar usuarios/roles inexistentes y duplicar la misma asignación.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de usuario_rol y por qué no usamos un dato visible del negocio como PK?
- ¿Por qué las columnas marcadas NOT NULL son obligatorias?
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

7. Tabla ESTACIONAMIENTO

Propósito: Instalación física administrada por ParkFlow.

```
CREATE TABLE estacionamiento (
    estacionamiento_id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    codigo              VARCHAR(30) NOT NULL,
    nombre              VARCHAR(120) NOT NULL,
    direccion           VARCHAR(220) NOT NULL,
    latitud             NUMERIC(9,6),
    longitud            NUMERIC(9,6),
    activo              BOOLEAN      NOT NULL,
    created_at          TIMESTAMPTZ NOT NULL,
    updated_at          TIMESTAMPTZ NOT NULL,

    CONSTRAINT pk_estacionamiento PRIMARY KEY (estacionamiento_id),
    CONSTRAINT uq_estacionamiento_codigo UNIQUE (codigo)
);

COMMENT ON TABLE estacionamiento IS 'Instalacion fisica de estacionamiento administrada por el sistema.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
estacionamiento_id	BIGINT IDENTITY	NO	PK	ID técnico.	Referencia desde zona y tarifa.
codigo	VARCHAR(30)	NO	UNIQUE	Código operativo.	No se repite en el sistema.
nombre	VARCHAR(120)	NO	-	Nombre visible.	Identifica la instalación al usuario.
direccion	VARCHAR(220)	NO	-	Ubicación textual.	Dato mínimo de localización.
latitud	NUMERIC(9,6)	SÍ	-	Coordenada opcional.	Puede no conocerse al crear el registro.
longitud	NUMERIC(9,6)	SÍ	-	Coordenada opcional.	Puede no conocerse al crear el registro.
activo	BOOLEAN	NO	-	Vigencia.	Permite inactivar sin borrar.
created_at	TIMESTAMPTZ	NO	-	Creación.	Trazabilidad.
updated_at	TIMESTAMPTZ	NO	-	Actualización.	Trazabilidad.

Constraints y relaciones que debes entender

- uq_estacionamiento_codigo protege un código globalmente único.
- ESTACIONAMIENTO es padre de ZONA y TARIFA.

¿Qué error evita?

Ejemplo de protección

Impide dos estacionamientos con el mismo código.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de estacionamiento y por qué no usamos un dato visible del negocio como PK?

- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Por qué esta tabla puede crearse antes que las tablas que la referencian?

8. Tabla ZONA

Propósito: Sector, piso, bloque o ala dentro de un estacionamiento.

```
CREATE TABLE zona (
    zona_id          BIGINT GENERATED BY DEFAULT AS IDENTITY,
    estacionamiento_id BIGINT      NOT NULL,
    codigo           VARCHAR(20)  NOT NULL,
    nombre           VARCHAR(100) NOT NULL,
    piso             VARCHAR(30),
    descripcion       TEXT,
    activo           BOOLEAN      NOT NULL,

    CONSTRAINT pk_zona PRIMARY KEY (zona_id),
    CONSTRAINT fk_zona_estacionamiento
        FOREIGN KEY (estacionamiento_id)
        REFERENCES estacionamiento (estacionamiento_id),
    CONSTRAINT uq_zona_estacionamiento_codigo
        UNIQUE (estacionamiento_id, codigo)
);

COMMENT ON TABLE zona IS 'Agrupar espacios dentro de un estacionamiento: piso, sector, ala o bloque.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
zona_id	BIGINT IDENTITY	NO	PK	ID técnico.	Identifica una zona.
estacionamiento_id	BIGINT	NO	FK -> estacionamiento	Estacionamiento padre.	Materializa 1:N estacionamiento-zona.
codigo	VARCHAR(20)	NO	UNIQUE compuesto	Código de zona dentro del estacionamiento.	Se combina con estacionamiento_id para unicidad contextual.
nombre	VARCHAR(100)	NO	-	Nombre visible.	Facilita operación.
piso	VARCHAR(30)	SÍ	-	Piso o nivel.	No toda zona necesita piso.
descripcion	TEXT	SÍ	-	Descripción.	Opcional.
activo	BOOLEAN	NO	-	Vigencia.	Permite inactivar sin borrar.

Constraints y relaciones que debes entender

- FK estacionamiento_id ubica ZONA en el lado N.
- UNIQUE(estacionamiento_id, codigo) es una unicidad contextual: el código puede repetirse en otro estacionamiento, pero no dentro del mismo.

¿Qué error evita?

Ejemplo de protección

Impide zonas huérfanas y códigos repetidos dentro del mismo estacionamiento.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de zona y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?

- ¿Qué relación del DER está materializando la FK principal de esta tabla?

9. Tabla ESPACIO

Propósito: Plaza física ocupable/reservable.

```
CREATE TABLE espacio (
    espacio_id      BIGINT GENERATED BY DEFAULT AS IDENTITY,
    zona_id         BIGINT      NOT NULL,
    codigo          VARCHAR(20) NOT NULL,
    tipo_vehiculo   VARCHAR(20) NOT NULL,
    estado          VARCHAR(20) NOT NULL,
    activo          BOOLEAN      NOT NULL,
    observacion     TEXT,
    created_at      TIMESTAMPTZ NOT NULL,
    updated_at      TIMESTAMPTZ NOT NULL,

    CONSTRAINT pk_espacio PRIMARY KEY (espacio_id),
    CONSTRAINT fk_espacio_zona
        FOREIGN KEY (zona_id)
        REFERENCES zona (zona_id),
    CONSTRAINT uq_espacio_zona_codigo
        UNIQUE (zona_id, codigo),
    CONSTRAINT ck_espacio_tipo_vehiculo
        CHECK (tipo_vehiculo IN ('AUTO', 'MOTO', 'SUV', 'CAMIONETA', 'VAN', 'OTRO')),
    CONSTRAINT ck_espacio_estado
        CHECK (estado IN ('DISPONIBLE', 'RESERVADO', 'OCUPADO', 'BLOQUEADO', 'FUERA_SERVICIO'))
);

COMMENT ON TABLE espacio IS 'Plaza física que puede estar disponible, reservada, ocupada o bloqueada.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
espacio_id	BIGINT IDENTITY	NO	PK	ID del espacio.	Referencia desde reserva, estancia, bloqueo e incidencia.
zona_id	BIGINT	NO	FK -> zona	Zona padre.	Materializa 1:N zona-espacio.
codigo	VARCHAR(20)	NO	UNIQUE compuesto	Código operativo.	Único dentro de la zona.
tipo_vehiculo	VARCHAR(20)	NO	CHECK	Tipo admitido.	Sólo acepta valores definidos.
estado	VARCHAR(20)	NO	CHECK	Estado operativo.	Sólo acepta DISPONIBLE/RESERVADO/OCUPADO/BLOQUEADO/FUERA_SERVICIO.
activo	BOOLEAN	NO	-	Vigencia.	Distingue vigencia del estado operativo.
observacion	TEXT	SÍ	-	Nota libre.	Opcional.
created_at	TIMESTAMPTZ	NO	-	Creación.	Trazabilidad.
updated_at	TIMESTAMPTZ	NO	-	Actualización.	Trazabilidad.

Constraints y relaciones que debes entender

- FK zona_id obliga a que el espacio pertenezca a una zona existente.
- UNIQUE(zona_id, codigo) evita dos espacios con el mismo código dentro de una zona.
- CHECK tipo_vehiculo y CHECK estado convierten dominios conceptuales en reglas ejecutables.

¿Qué error evita?

Ejemplo de protección

Impide espacios huérfanos, códigos duplicados por zona y estados/tipos inválidos.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de espacio y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

10. Tabla CLIENTE

Propósito: Persona propietaria o usuaria de vehículos.

```
CREATE TABLE cliente (
    cliente_id          BIGINT GENERATED BY DEFAULT AS IDENTITY,
    numero_documento    VARCHAR(40) NOT NULL,
    nombre_completo     VARCHAR(160) NOT NULL,
    email               VARCHAR(160),
    telefono            VARCHAR(40),
    fecha_nacimiento    DATE,
    activo              BOOLEAN      NOT NULL,
    created_at          TIMESTAMPTZ NOT NULL,

    CONSTRAINT pk_cliente PRIMARY KEY (cliente_id),
    CONSTRAINT uq_cliente_numero_documento UNIQUE (numero_documento),
    CONSTRAINT uq_cliente_email UNIQUE (email)
);

COMMENT ON TABLE cliente IS 'Persona propietaria o usuaria de los vehiculos registrados.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
cliente_id	BIGINT IDENTITY	NO	PK	ID del cliente.	Referencia desde vehículo.
numero_documento	VARCHAR(40)	NO	UNIQUE	Documento.	Evita duplicar clientes por documento.
nombre_completo	VARCHAR(160)	NO	-	Nombre.	Dato obligatorio de identificación.
email	VARCHAR(160)	SÍ	UNIQUE	Correo.	Opcional pero, si existe, no se repite.
telefono	VARCHAR(40)	SÍ	-	Contacto.	Opcional.
fecha_nacimiento	DATE	SÍ	-	Nacimiento.	Sólo interesa la fecha, no la hora.
activo	BOOLEAN	NO	-	Vigencia.	Permite inactivar.
created_at	TIMESTAMPTZ	NO	-	Creación.	Trazabilidad.

Constraints y relaciones que debes entender

- numero_documento es UNIQUE global.
- email es opcional y UNIQUE; PostgreSQL permite varios NULL en un UNIQUE tradicional.
- No hay FK porque el cliente es raíz del subdominio Cliente-Vehículo.

¿Qué error evita?

Ejemplo de protección

Impide duplicar documento y, cuando exista, email.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de cliente y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?

- ¿Por qué esta tabla puede crearse antes que las tablas que la referencian?

11. Tabla VEHICULO

Propósito: Vehículo asociado a un cliente.

```
CREATE TABLE vehiculo (
    vehiculo_id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    cliente_id BIGINT NOT NULL,
    placa VARCHAR(20) NOT NULL,
    tipo VARCHAR(20) NOT NULL,
    marca VARCHAR(60),
    modelo VARCHAR(60),
    color VARCHAR(40),
    activo BOOLEAN NOT NULL,

    CONSTRAINT pk_vehiculo PRIMARY KEY (vehiculo_id),
    CONSTRAINT fk_vehiculo_cliente
        FOREIGN KEY (cliente_id)
        REFERENCES cliente (cliente_id),
    CONSTRAINT uq_vehiculo_placa UNIQUE (placa),
    CONSTRAINT ck_vehiculo_tipo
        CHECK (tipo IN ('AUTO', 'MOTO', 'SUV', 'CAMIONETA', 'VAN', 'OTRO'))
);

COMMENT ON TABLE vehiculo IS 'Vehiculo registrado y asociado a un cliente.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
vehiculo_id	BIGINT IDENTITY	NO	PK	ID técnico.	Referencia estable.
cliente_id	BIGINT	NO	FK -> cliente	Propietario/cliente.	No puede existir sin cliente en esta V1.
placa	VARCHAR(20)	NO	UNIQUE	Placa.	Evita registrar dos veces el mismo vehículo.
tipo	VARCHAR(20)	NO	CHECK	Tipo de vehículo.	Dominio controlado.
marca	VARCHAR(60)	SÍ	-	Marca.	Opcional.
modelo	VARCHAR(60)	SÍ	-	Modelo.	Opcional.
color	VARCHAR(40)	SÍ	-	Color.	Opcional.
activo	BOOLEAN	NO	-	Vigencia.	Permite inactivar.

Constraints y relaciones que debes entender

- FK cliente_id expresa Cliente 1:N Vehículo.
- placa UNIQUE evita duplicidad física/operativa.
- CHECK tipo restringe el conjunto de valores.

¿Qué error evita?

Ejemplo de protección

Impide vehículos sin cliente, placas duplicadas y tipos inválidos.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de vehiculo y por qué no usamos un dato visible del negocio como PK?

- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

12. Tabla TARIFA

Propósito: Regla de cobro aplicable a una estancia.

```
CREATE TABLE tarifa (
    tarifa_id          BIGINT GENERATED BY DEFAULT AS IDENTITY,
    estacionamiento_id BIGINT          NOT NULL,
    nombre             VARCHAR(100)   NOT NULL,
    tipo_vehiculo      VARCHAR(20),
    monto              NUMERIC(12,2)  NOT NULL,
    unidad             VARCHAR(20)    NOT NULL,
    vigente_desde      TIMESTAMPTZ    NOT NULL,
    vigente_hasta      TIMESTAMPTZ,
    activo             BOOLEAN        NOT NULL,

    CONSTRAINT pk_tarifa PRIMARY KEY (tarifa_id),
    CONSTRAINT fk_tarifa_estacionamiento
        FOREIGN KEY (estacionamiento_id)
        REFERENCES estacionamiento (estacionamiento_id),
    CONSTRAINT ck_tarifa_tipo_vehiculo
        CHECK (tipo_vehiculo IS NULL OR tipo_vehiculo IN ('AUTO', 'MOTO', 'SUV', 'CAMIONETA', 'VAN',
'OTRO')),
    CONSTRAINT ck_tarifa_monto
        CHECK (monto >= 0),
    CONSTRAINT ck_tarifa_unidad
        CHECK (unidad IN ('HORA', 'FRACCION', 'DIA', 'ESTANCIA', 'OTRO')),
    CONSTRAINT ck_tarifa_vigencia
        CHECK (vigente_hasta IS NULL OR vigente_hasta > vigente_desde)
);

COMMENT ON TABLE tarifa IS 'Reglas de cobro que pueden aplicarse a una estancia.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
tarifa_id	BIGINT IDENTITY	NO	PK	ID técnico.	Referencia desde estancia.
estacionamiento_id	BIGINT	NO	FK -> estacionamiento	Instalación a la que pertenece.	Una tarifa pertenece a un estacionamiento.
nombre	VARCHAR(100)	NO	-	Nombre.	Identifica la tarifa.
tipo_vehiculo	VARCHAR(20)	SÍ	CHECK	Tipo al que aplica.	NULL puede representar una tarifa general.
monto	NUMERIC(12,2)	NO	CHECK >= 0	Importe.	Evita montos negativos y conserva exactitud decimal.
unidad	VARCHAR(20)	NO	CHECK	Unidad de cobro.	Sólo valores HORA/FRACCION/DIA/ESTANCIA/OTRO.
vigente_desde	TIMESTAMPTZ	NO	-	Inicio de vigencia.	Momento exacto.
vigente_hasta	TIMESTAMPTZ	SÍ	CHECK	Fin de vigencia.	Si existe debe ser posterior al inicio.
activo	BOOLEAN	NO	-	Vigencia lógica.	Permite retirar tarifa.

Constraints y relaciones que debes entender

- FK estacionamiento_id indica a qué instalación pertenece la tarifa.
- CHECK monto evita valores negativos.
- CHECK unidad controla HORA/FRACCION/DIA/ESTANCIA/OTRO.

- CHECK vigencia garantiza que vigente_hasta, si existe, sea posterior a vigente_desde.

¿Qué error evita?

Ejemplo de protección

Impide tarifas huérfanas, montos negativos, unidades inválidas y vigencias invertidas.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de tarifa y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

13. Tabla RESERVA

Propósito: Aparta un espacio para un vehículo durante un intervalo.

```
CREATE TABLE reserva (
    reserva_id          BIGINT GENERATED BY DEFAULT AS IDENTITY,
    vehiculo_id         BIGINT      NOT NULL,
    espacio_id          BIGINT      NOT NULL,
    fecha_inicio        TIMESTAMPTZ NOT NULL,
    fecha_fin           TIMESTAMPTZ NOT NULL,
    estado              VARCHAR(20) NOT NULL,
    motivo_cancelacion  TEXT,
    created_at          TIMESTAMPTZ NOT NULL,
    updated_at          TIMESTAMPTZ NOT NULL,

    CONSTRAINT pk_reserva PRIMARY KEY (reserva_id),
    CONSTRAINT fk_reserva_vehiculo
        FOREIGN KEY (vehiculo_id)
        REFERENCES vehiculo (vehiculo_id),
    CONSTRAINT fk_reserva_espacio
        FOREIGN KEY (espacio_id)
        REFERENCES espacio (espacio_id),
    CONSTRAINT ck_reserva_intervalo
        CHECK (fecha_fin > fecha_inicio),
    CONSTRAINT ck_reserva_estado
        CHECK (estado IN ('PENDIENTE', 'CONFIRMADA', 'CANCELADA', 'VENCIDA', 'USADA')),
    CONSTRAINT ck_reserva_motivo_cancelacion
        CHECK (
            estado <> 'CANCELADA'
            OR (motivo_cancelacion IS NOT NULL AND BTRIM(motivo_cancelacion) <> '')
        )
);

COMMENT ON TABLE reserva IS 'Intencion de apartar un espacio para un vehiculo durante un intervalo.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
reserva_id	BIGINT IDENTITY	NO	PK	ID técnico.	Referencia desde estancia.
vehiculo_id	BIGINT	NO	FK -> vehiculo	Vehículo reservado.	Debe existir.
espacio_id	BIGINT	NO	FK -> espacio	Espacio reservado.	Debe existir.
fecha_inicio	TIMESTAMPTZ	NO	CHECK intervalo	Inicio.	Momento exacto.
fecha_fin	TIMESTAMPTZ	NO	CHECK intervalo	Fin.	Debe ser posterior a inicio.
estado	VARCHAR(20)	NO	CHECK	Estado de reserva.	Dominio cerrado.
motivo_cancelacion	TEXT	SÍ	CHECK condicional	Motivo.	Obligatorio sólo si estado=CANCELADA.
created_at	TIMESTAMPTZ	NO	-	Creación.	Trazabilidad.
updated_at	TIMESTAMPTZ	NO	-	Actualización.	Trazabilidad.

Constraints y relaciones que debes entender

- FK vehiculo_id y espacio_id obligan a referenciar filas reales.
- CHECK fecha_fin > fecha_inicio protege coherencia temporal.

- CHECK estado controla el ciclo de vida permitido.
- CHECK motivo_cancelacion expresa una regla condicional dentro de la misma fila.
- Evitar solapamientos entre reservas requiere lógica entre varias filas y no está resuelto por un CHECK simple.

¿Qué error evita?

Ejemplo de protección

Impide reservas con vehículo/espacio inexistente, intervalos invertidos, estados inválidos y cancelaciones sin motivo.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de reserva y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

14. Tabla ESTANCIA

Propósito: Ocupación real de un espacio por un vehículo.

```
CREATE TABLE estancia (
    estancia_id    BIGINT GENERATED BY DEFAULT AS IDENTITY,
    vehiculo_id    BIGINT          NOT NULL,
    espacio_id     BIGINT          NOT NULL,
    reserva_id     BIGINT,
    tarifa_id      BIGINT,
    check_in_at    TIMESTAMPTZ    NOT NULL,
    check_out_at   TIMESTAMPTZ,
    estado         VARCHAR(20)    NOT NULL,
    monto          NUMERIC(12,2),
    created_at     TIMESTAMPTZ    NOT NULL,

    CONSTRAINT pk_estancia PRIMARY KEY (estancia_id),
    CONSTRAINT fk_estancia_vehiculo
        FOREIGN KEY (vehiculo_id)
        REFERENCES vehiculo (vehiculo_id),
    CONSTRAINT fk_estancia_espacio
        FOREIGN KEY (espacio_id)
        REFERENCES espacio (espacio_id),
    CONSTRAINT fk_estancia_reserva
        FOREIGN KEY (reserva_id)
        REFERENCES reserva (reserva_id),
    CONSTRAINT fk_estancia_tarifa
        FOREIGN KEY (tarifa_id)
        REFERENCES tarifa (tarifa_id),
    CONSTRAINT uq_estancia_reserva UNIQUE (reserva_id),
    CONSTRAINT ck_estancia_salida
        CHECK (check_out_at IS NULL OR check_out_at > check_in_at),
    CONSTRAINT ck_estancia_estado
        CHECK (estado IN ('ABIERTA', 'CERRADA', 'CANCELADA')),
    CONSTRAINT ck_estancia_monto
        CHECK (monto IS NULL OR monto >= 0)
);

COMMENT ON TABLE estancia IS 'Ingreso real de un vehiculo y ocupacion efectiva de un espacio.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
estancia_id	BIGINT IDENTITY	NO	PK	ID técnico.	Referencia desde pago e incidencia.
vehiculo_id	BIGINT	NO	FK -> vehiculo	Vehículo que ingresó.	Debe existir.
espacio_id	BIGINT	NO	FK -> espacio	Espacio ocupado.	Debe existir.
reserva_id	BIGINT	SÍ	FK + UNIQUE	Reserva de origen.	Opcional; UNIQUE implementa Reserva 1:0..1 Estancia.
tarifa_id	BIGINT	SÍ	FK -> tarifa	Tarifa aplicada.	Puede asignarse después.
check_in_at	TIMESTAMPTZ	NO	-	Ingreso real.	Momento obligatorio.

check_out_at	TIMESTAMPTZ	SÍ	CHECK	Salida.	Mientras está abierta puede ser NULL; si existe > ingreso.
estado	VARCHAR(20)	NO	CHECK	ABIERTA/CERRADA/CANCELADA.	Evita estados inventados.
monto	NUMERIC(12,2)	SÍ	CHECK >= 0	Monto calculado.	Puede no conocerse mientras está abierta.
created_at	TIMESTAMPTZ	NO	-	Creación.	Trazabilidad.

Constraints y relaciones que debes entender

- FK vehiculo, espacio, reserva y tarifa conectan la estancia con su contexto.
- reserva_id es nullable porque puede existir ingreso sin reserva.
- UNIQUE(reserva_id) impide que una misma reserva produzca dos estancias y materializa 1:0..1.
- CHECK salida y monto protegen coherencia temporal/económica.

¿Qué error evita?

Ejemplo de protección

Impide estancias con referencias inexistentes, una reserva usada por dos estancias, salida anterior al ingreso y montos negativos.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de estancia y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

15. Tabla PAGO_OPERATIVO

Propósito: Registro de pago asociado a una estancia.

```
CREATE TABLE pago_operativo (
    pago_id          BIGINT GENERATED BY DEFAULT AS IDENTITY,
    estancia_id      BIGINT          NOT NULL,
    monto            NUMERIC(12,2)  NOT NULL,
    metodo           VARCHAR(30)    NOT NULL,
    referencia       VARCHAR(100),
    paid_at          TIMESTAMPTZ    NOT NULL,
    estado           VARCHAR(20)    NOT NULL,

    CONSTRAINT pk_pago_operativo PRIMARY KEY (pago_id),
    CONSTRAINT fk_pago_operativo_estancia
        FOREIGN KEY (estancia_id)
        REFERENCES estancia (estancia_id),
    CONSTRAINT ck_pago_operativo_monto
        CHECK (monto >= 0),
    CONSTRAINT ck_pago_operativo_metodo
        CHECK (metodo IN ('EFFECTIVO', 'QR', 'TARJETA', 'TRANSFERENCIA', 'OTRO')),
    CONSTRAINT ck_pago_operativo_estado
        CHECK (estado IN ('PENDIENTE', 'CONFIRMADO', 'ANULADO'))
);

COMMENT ON TABLE pago_operativo IS 'Pago operativo asociado a una estancia; no representa una pasarela bancaria real.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
pago_id	BIGINT IDENTITY	NO	PK	ID del pago.	Identifica cada operación.
estancia_id	BIGINT	NO	FK -> estancia	Estancia pagada.	No existe pago sin estancia.
monto	NUMERIC(12,2)	NO	CHECK >= 0	Monto pagado.	No puede ser negativo.
metodo	VARCHAR(30)	NO	CHECK	Medio de pago.	Dominio controlado.
referencia	VARCHAR(100)	SÍ	-	Referencia externa.	Puede no existir, p.ej. efectivo.
paid_at	TIMESTAMPTZ	NO	-	Momento del pago.	Trazabilidad.
estado	VARCHAR(20)	NO	CHECK	PENDIENTE/CONFIRMADO/ANULADO.	Evita estados libres.

Constraints y relaciones que debes entender

- FK estancia_id obliga a que todo pago corresponda a una estancia existente.
- CHECK monto impide negativos.
- CHECK metodo y estado evitan texto libre inconsistente.

¿Qué error evita?

Ejemplo de protección

Impide pagos sin estancia, montos negativos y valores no admitidos de método/estado.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de pago_operativo y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

16. Tabla BLOQUEO_ESPACIO

Propósito: Periodo en que un espacio no debe estar disponible.

```
CREATE TABLE bloqueo_espacio (
    bloqueo_id      BIGINT GENERATED BY DEFAULT AS IDENTITY,
    espacio_id      BIGINT      NOT NULL,
    fecha_inicio    TIMESTAMPTZ NOT NULL,
    fecha_fin       TIMESTAMPTZ,
    motivo          VARCHAR(200) NOT NULL,
    estado          VARCHAR(20)  NOT NULL,
    created_by      BIGINT      NOT NULL,
    created_at      TIMESTAMPTZ NOT NULL,

    CONSTRAINT pk_bloqueo_espacio PRIMARY KEY (bloqueo_id),
    CONSTRAINT fk_bloqueo_espacio_espacio
        FOREIGN KEY (espacio_id)
        REFERENCES espacio (espacio_id),
    CONSTRAINT fk_bloqueo_espacio_usuario
        FOREIGN KEY (created_by)
        REFERENCES usuario (usuario_id),
    CONSTRAINT ck_bloqueo_espacio_intervalo
        CHECK (fecha_fin IS NULL OR fecha_fin > fecha_inicio),
    CONSTRAINT ck_bloqueo_espacio_estado
        CHECK (estado IN ('PROGRAMADO', 'ACTIVO', 'FINALIZADO', 'CANCELADO'))
);

COMMENT ON TABLE bloqueo_espacio IS 'Periodo durante el cual un espacio no debe estar disponible para uso normal.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
bloqueo_id	BIGINT IDENTITY	NO	PK	ID del bloqueo.	Identifica el evento.
espacio_id	BIGINT	NO	FK -> espacio	Espacio bloqueado.	Debe existir.
fecha_inicio	TIMESTAMPTZ	NO	CHECK intervalo	Inicio.	Obligatorio.
fecha_fin	TIMESTAMPTZ	SÍ	CHECK intervalo	Fin.	Puede quedar abierto; si existe > inicio.
motivo	VARCHAR(200)	NO	-	Razón del bloqueo.	Obligatoria para trazabilidad.
estado	VARCHAR(20)	NO	CHECK	Estado del bloqueo.	Dominio controlado.
created_by	BIGINT	NO	FK -> usuario	Usuario que creó.	Responsabilidad.
created_at	TIMESTAMPTZ	NO	-	Creación.	Trazabilidad.

Constraints y relaciones que debes entender

- FK espacio_id conecta el bloqueo con una plaza real.
- FK created_by registra responsabilidad del usuario.
- CHECK intervalo permite bloqueo abierto con fecha_fin NULL, pero si existe debe ser posterior.
- CHECK estado controla el ciclo del bloqueo.

¿Qué error evita?

Ejemplo de protección

Impide bloquear espacios inexistentes, registrar creador inexistente, intervalos inválidos y estados no reconocidos.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de bloqueo_espacio y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

17. Tabla INCIDENCIA

Propósito: Problema o evento excepcional trazable.

```
CREATE TABLE incidencia (
    incidencia_id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    espacio_id BIGINT,
    estancia_id BIGINT,
    tipo VARCHAR(40) NOT NULL,
    descripcion TEXT NOT NULL,
    estado VARCHAR(20) NOT NULL,
    created_by BIGINT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL,
    resolved_at TIMESTAMPTZ,

    CONSTRAINT pk_incidencia PRIMARY KEY (incidencia_id),
    CONSTRAINT fk_incidencia_espacio
        FOREIGN KEY (espacio_id)
        REFERENCES espacio (espacio_id),
    CONSTRAINT fk_incidencia_estancia
        FOREIGN KEY (estancia_id)
        REFERENCES estancia (estancia_id),
    CONSTRAINT fk_incidencia_usuario
        FOREIGN KEY (created_by)
        REFERENCES usuario (usuario_id),
    CONSTRAINT ck_incidencia_tipo_no_vacio
        CHECK (BTRIM(tipo) <> ''),
    CONSTRAINT ck_incidencia_estado
        CHECK (estado IN ('ABIERTA', 'EN_REVISION', 'RESUELTA', 'CERRADA')),
    CONSTRAINT ck_incidencia_resolved_at
        CHECK (resolved_at IS NULL OR resolved_at >= created_at)
);

COMMENT ON TABLE incidencia IS 'Problema o evento excepcional que debe ser atendido y trazado.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
incidencia_id	BIGINT IDENTITY	NO	PK	ID de incidencia.	Identifica el caso.
espacio_id	BIGINT	SÍ	FK -> espacio	Espacio relacionado.	Opcional.
estancia_id	BIGINT	SÍ	FK -> estancia	Estancia relacionada.	Opcional.
tipo	VARCHAR(40)	NO	CHECK no vacío	Tipo de incidencia.	No se fija catálogo cerrado porque el modelo no lo define.
descripcion	TEXT	NO	-	Detalle.	Obligatorio.
estado	VARCHAR(20)	NO	CHECK	Estado.	Dominio controlado.
created_by	BIGINT	NO	FK -> usuario	Usuario responsable del registro.	Trazabilidad.
created_at	TIMESTAMPTZ	NO	-	Creación.	Trazabilidad.
resolved_at	TIMESTAMPTZ	SÍ	CHECK	Resolución.	Si existe no puede ser anterior a creación.

Constraints y relaciones que debes entender

- espacio_id y estancia_id son FK opcionales: una incidencia puede asociarse a uno de esos contextos.
- created_by es obligatorio para trazabilidad.
- tipo no usa catálogo cerrado porque el modelo fuente no definía uno exhaustivo.
- resolved_at no puede ser anterior a created_at.

¿Qué error evita?

Ejemplo de protección

Impide referencias inválidas, tipos vacíos, estados inválidos y resoluciones cronológicamente imposibles.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de incidencia y por qué no usamos un dato visible del negocio como PK?
- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

18. Tabla HISTORIAL_ESTADO

Propósito: Registro cronológico de cambios de estado.

```
CREATE TABLE historial_estado (
    historial_estado_id BIGINT GENERATED BY DEFAULT AS IDENTITY,
    resource_type        VARCHAR(50) NOT NULL,
    resource_id          BIGINT      NOT NULL,
    estado_anterior      VARCHAR(30),
    estado_nuevo         VARCHAR(30) NOT NULL,
    changed_by          BIGINT      NOT NULL,
    changed_at           TIMESTAMPTZ NOT NULL,
    motivo              TEXT,

    CONSTRAINT pk_historial_estado PRIMARY KEY (historial_estado_id),
    CONSTRAINT fk_historial_estado_usuario
        FOREIGN KEY (changed_by)
        REFERENCES usuario (usuario_id)
);

COMMENT ON TABLE historial_estado IS 'Registro cronologico de cambios de estado de recursos del sistema.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
historial_estado_id	BIGINT IDENTITY	NO	PK	ID del evento.	Identifica cada cambio.
resource_type	VARCHAR(50)	NO	Referencia lógica	Tipo de recurso.	Permite historial de diferentes tablas.
resource_id	BIGINT	NO	Referencia lógica	ID del recurso.	No es FK física porque el destino depende de resource_type.
estado_anterior	VARCHAR(30)	SÍ	-	Estado previo.	Puede ser NULL en el primer estado.
estado_nuevo	VARCHAR(30)	NO	-	Nuevo estado.	Obligatorio.
changed_by	BIGINT	NO	FK -> usuario	Quién hizo el cambio.	Trazabilidad.
changed_at	TIMESTAMPTZ	NO	-	Cuándo.	Orden cronológico.
motivo	TEXT	SÍ	-	Razón.	Opcional.

Constraints y relaciones que debes entender

- changed_by es FK física a usuario.
- resource_type + resource_id forman una referencia lógica polimórfica.
- resource_id no puede tener una sola FK porque unas filas pueden representar reservas, otras espacios, otras estancias, etc.

¿Qué error evita?

Ejemplo de protección

Impide atribuir un cambio a un usuario inexistente, conservando flexibilidad para múltiples tipos de recurso.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de historial_estado y por qué no usamos un dato visible del negocio como PK?

- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

19. Tabla AUDITORIA

Propósito: Registro técnico/funcional de acciones relevantes.

```
CREATE TABLE auditoria (
    auditoria_id    BIGINT GENERATED BY DEFAULT AS IDENTITY,
    usuario_id      BIGINT,
    accion          VARCHAR(60) NOT NULL,
    resource_type   VARCHAR(60) NOT NULL,
    resource_id     BIGINT,
    occurred_at     TIMESTAMPTZ NOT NULL,
    ip_address      VARCHAR(64),
    details         JSONB,

    CONSTRAINT pk_auditoria PRIMARY KEY (auditoria_id),
    CONSTRAINT fk_auditoria_usuario
        FOREIGN KEY (usuario_id)
        REFERENCES usuario (usuario_id)
);

COMMENT ON TABLE auditoria IS 'Eventos tecnicos o funcionales relevantes para trazabilidad.';
```

Lectura atributo por atributo

Columna	Tipo	NULL	Clave / restricción	Qué representa	Por qué importa
auditoria_id	BIGINT IDENTITY	NO	PK	ID del evento.	Identifica el registro.
usuario_id	BIGINT	SÍ	FK -> usuario	Usuario si existe.	Puede ser NULL para procesos técnicos.
accion	VARCHAR(60)	NO	-	Acción realizada.	Describe el evento.
resource_type	VARCHAR(60)	NO	Referencia lógica	Tipo de recurso.	Permite diferentes recursos.
resource_id	BIGINT	SÍ	Referencia lógica	ID del recurso.	Puede no aplicar en eventos globales.
occurred_at	TIMESTAMPTZ	NO	-	Momento.	Trazabilidad.
ip_address	VARCHAR(64)	SÍ	-	IP origen.	Opcional.
details	JSONB	SÍ	-	Detalles variables.	Permite contexto flexible sin alterar el esquema por cada tipo de evento.

Constraints y relaciones que debes entender

- usuario_id es FK opcional porque algunos eventos pueden ser técnicos/sin usuario humano.
- resource_type + resource_id es referencia lógica polimórfica.
- JSONB permite guardar detalles variables de auditoría sin crear una estructura distinta para cada acción.

¿Qué error evita?

Ejemplo de protección

Impide referenciar un usuario inexistente cuando hay usuario, y permite auditar procesos técnicos con usuario NULL.

Preguntas para comprobar comprensión

- ¿Cuál es la PK de auditoria y por qué no usamos un dato visible del negocio como PK?

- Muestra una columna nullable y explica qué estado válido representa su ausencia.
- ¿Qué constraint de esta tabla consideras más importante y qué dato inválido bloquearía?
- ¿Qué relación del DER está materializando la FK principal de esta tabla?

20. Índices recomendados

```

CREATE INDEX ix_usuario_rol_usuario_id
    ON usuario_rol (usuario_id);
CREATE INDEX ix_usuario_rol_rol_id
    ON usuario_rol (rol_id);

CREATE INDEX ix_zona_estacionamiento_id
    ON zona (estacionamiento_id);

CREATE INDEX ix_espacio_zona_id
    ON espacio (zona_id);
CREATE INDEX ix_espacio_estado
    ON espacio (estado);

CREATE INDEX ix_vehiculo_cliente_id
    ON vehiculo (cliente_id);

CREATE INDEX ix_tarifa_estacionamiento_id
    ON tarifa (estacionamiento_id);

CREATE INDEX ix_reserva_vehiculo_id
    ON reserva (vehiculo_id);
CREATE INDEX ix_reserva_espacio_id
    ON reserva (espacio_id);
CREATE INDEX ix_reserva_intervalo
    ON reserva (espacio_id, fecha_inicio, fecha_fin);
CREATE INDEX ix_reserva_estado
    ON reserva (estado);

CREATE INDEX ix_estancia_vehiculo_id
    ON estancia (vehiculo_id);
CREATE INDEX ix_estancia_espacio_id
    ON estancia (espacio_id);
CREATE INDEX ix_estancia_tarifa_id
    ON estancia (tarifa_id);
CREATE INDEX ix_estancia_estado
    ON estancia (estado);

CREATE INDEX ix_pago_operativo_estancia_id
    ON pago_operativo (estancia_id);

CREATE INDEX ix_bloqueo_espacio_espacio_id
    ON bloqueo_espacio (espacio_id);
CREATE INDEX ix_bloqueo_espacio_created_by
    ON bloqueo_espacio (created_by);
CREATE INDEX ix_bloqueo_espacio_intervalo
    ON bloqueo_espacio (espacio_id, fecha_inicio, fecha_fin);

CREATE INDEX ix_incidencia_espacio_id
    ON incidencia (espacio_id);
CREATE INDEX ix_incidencia_estancia_id
    ON incidencia (estancia_id);
CREATE INDEX ix_incidencia_created_by
    ON incidencia (created_by);
CREATE INDEX ix_incidencia_estado

```

```

ON incidencia (estado);

CREATE INDEX ix_historial_estado_resource
  ON historial_estado (resource_type, resource_id, changed_at);
CREATE INDEX ix_historial_estado_changed_by
  ON historial_estado (changed_by);

CREATE INDEX ix_auditoria_usuario_id
  ON auditoria (usuario_id);
CREATE INDEX ix_auditoria_resource
  ON auditoria (resource_type, resource_id, occurred_at);

```

- Un índice no cambia la integridad lógica: mejora la velocidad de búsqueda y JOIN.
- PostgreSQL crea automáticamente índice para PRIMARY KEY y UNIQUE, pero no crea automáticamente un índice por cada FOREIGN KEY.
- Por eso se crean índices sobre usuario_id, rol_id, estacionamiento_id, zona_id, cliente_id, etc.
- Los índices compuestos como ix_reserva_intervalo ayudan a consultas que buscan reservas por espacio y rango temporal.
- Los índices de resource_type + resource_id + fecha ayudan a consultar cronológicamente historial y auditoría por recurso.

No confundas

PRIMARY KEY/UNIQUE protegen reglas y también generan índices. Un CREATE INDEX normal busca rendimiento; no impide duplicados salvo que se declare UNIQUE INDEX.

21. Verificación final del script

```

SELECT table_schema, table_name
FROM information_schema.tables
WHERE table_schema = 'parkflow'
  AND table_type = 'BASE TABLE'
ORDER BY table_name;

```

- information_schema.tables es un catálogo estándar que permite consultar las tablas visibles.
- table_schema = 'parkflow' limita el resultado al schema del proyecto.
- table_type = 'BASE TABLE' excluye vistas y otros objetos.
- ORDER BY table_name ordena el resultado para revisarlo fácilmente.
- El resultado esperado es una lista con las 16 tablas principales.

22. Por qué el orden de creación importa

1. usuario
2. rol
3. usuario_rol
4. estacionamiento
5. zona
6. espacio
7. cliente
8. vehiculo
9. tarifa
10. reserva
11. estancia
12. pago_operativo

- 13. 13. bloqueo_espacio
- 14. 14. incidencia
- 15. 15. historial_estado
- 16. 16. auditoria

Regla práctica

Una tabla que contiene una FK normalmente debe crearse después de la tabla que referencia. Por ejemplo: no podemos crear ZONA con una FK a ESTACIONAMIENTO si ESTACIONAMIENTO todavía no existe.

usuario	raíz para usuario_rol, bloqueo_espacio, incidencia, historial_estado, auditoria
rol	raíz para usuario_rol
estacionamiento	raíz para zona y tarifa
zona	depende de estacionamiento; padre de espacio
espacio	depende de zona; padre de reserva, estancia, bloqueo, incidencia
cliente	raíz para vehiculo
vehiculo	depende de cliente; padre de reserva/estancia
tarifa	depende de estacionamiento; padre opcional de estancia
reserva	depende de vehiculo y espacio; padre opcional 1:0..1 de estancia
estancia	depende de vehiculo, espacio y opcionalmente reserva/tarifa; padre de pago e incidencia

23. Mapa rápido de constraints del proyecto

Mecanismo	Qué protege	Ejemplo ParkFlow	Error que bloquea
PRIMARY KEY	Identidad	usuario.usuario_id	Dos filas con el mismo ID / ID nulo
FOREIGN KEY	Referencia	vehiculo.cliente_id	Vehículo apuntando a cliente inexistente
UNIQUE	No repetición	vehiculo.placa	Dos vehículos con la misma placa
UNIQUE compuesta	No repetición contextual	zona(estacionamiento_id,codigo)	Dos zonas A en el mismo estacionamiento
NOT NULL	Obligatoriedad	reserva.fecha_inicio	Reserva sin fecha de inicio
CHECK	Coherencia de fila	reserva.fecha_fin > fecha_inicio	Intervalo invertido
CHECK con dominio	Valores permitidos	espacio.estado	Estado inventado
CHECK condicional	Regla dentro de fila	cancelada requiere motivo	Cancelación sin motivo
INDEX	Rendimiento	ix_reserva_intervalo	No bloquea error; acelera búsqueda

24. Reglas que este V1 todavía NO resuelve por sí solo

- Evitar dos reservas activas solapadas para el mismo espacio: requiere revisar otras filas; no es un CHECK simple.
- Cambiar automáticamente estado del espacio al confirmar reserva, abrir estancia o cerrar estancia: requiere lógica de aplicación/transacción.
- Calcular el monto final de una estancia según tarifa y duración: requiere lógica posterior.
- Controlar permisos por rol: las tablas permiten asignar roles, pero la autorización se implementará en backend.
- Garantizar que una incidencia tenga al menos espacio_id o estancia_id si esa regla se decide formalmente: el script actual no impone esa condición.
- Validar transiciones de estados (por ejemplo, de PENDIENTE a CONFIRMADA) según estado anterior: requiere conocer otra versión/estado, no sólo la fila nueva.

Concepto clave

Una base de datos debe proteger todo lo que pueda proteger de forma declarativa; pero no todas las reglas de negocio caben en PK/FK/UNIQUE/CHECK. Saber distinguirlas es parte del diseño.

25. Cómo ejecutarlo correctamente en DataGrip

17. Conéctate a la base de datos PostgreSQL que usarás para ParkFlow.
18. Abre una consola SQL asociada a esa conexión/base.
19. Pega o abre ParkFlow360_V1_Creacion_Completa_PostgreSQL.sql.
20. Revisa que no estés ejecutando contra otra base por error.
21. Ejecuta el script completo en orden, no bloques aleatorios.
22. Observa el panel de salida: si un constraint falla durante CREATE TABLE, detén la ejecución y lee el nombre del constraint.
23. Refresca el schema parkflow en DataGrip.
24. Ejecuta la consulta final de information_schema y confirma las 16 tablas.
25. No edites silenciosamente una V1 ya aplicada en un entorno compartido; una evolución real se modelará como una migración nueva.

26. Ejercicios de comprensión antes de insertar datos

26. Dibuja de memoria la cadena ESTACIONAMIENTO → ZONA → ESPACIO y escribe dónde vive cada FK.
27. Explica por qué usuario_rol necesita dos FK y un UNIQUE compuesto.
28. Explica por qué cliente.email puede ser NULL y a la vez UNIQUE.
29. Explica por qué reserva.fecha_fin > reserva.fecha_inicio sí puede ser CHECK, pero el solapamiento entre dos reservas no.
30. Explica cómo UNIQUE(reserva_id) en estancia implementa una relación 1:0..1.
31. Busca tres CHECK de dominio y escribe un INSERT inválido que cada uno rechazaría.
32. Busca cinco FK y di qué tabla debe existir antes para poder crearlas.
33. Explica por qué historial_estado.resource_id no tiene FK física.
34. Explica la diferencia entre historial_estado y auditoria.
35. Explica por qué agregamos índices a varias FK.

27. Banco de preguntas de defensa individual

- ¿Qué hace GENERATED BY DEFAULT AS IDENTITY?

- ¿Qué diferencia existe entre PRIMARY KEY y UNIQUE?
- ¿Qué diferencia existe entre UNIQUE global y UNIQUE compuesta/contextual?
- ¿Por qué la FK suele estar en el lado N de una relación 1:N?
- ¿Qué pasa si intento insertar un vehiculo con cliente_id que no existe?
- ¿Por qué email puede ser NULL pero seguir teniendo UNIQUE?
- ¿Qué constraint evita una reserva con fecha final anterior a la inicial?
- ¿Qué constraint impide que una reserva cancelada carezca de motivo?
- ¿Por qué reserva_id en estancia tiene FK + UNIQUE?
- ¿Por qué resource_id no tiene FK en historial_estado?
- ¿Para qué sirve JSONB en auditoria?
- ¿Qué diferencia hay entre un constraint y un índice?
- ¿PostgreSQL crea automáticamente índices para FK?
- ¿Qué regla importante de reservas queda fuera de este V1 y por qué?
- Si elimino uq_vehiculo_placa, ¿qué estado inconsistente podría aparecer?

28. Definition of Done para estudiar este script

- Puedo ejecutar el script completo sin depender de otra persona.
- Puedo explicar qué crea el schema parkflow.
- Puedo identificar la PK de las 16 tablas.
- Puedo seguir todas las FK principales en el DER.
- Puedo explicar al menos tres UNIQUE simples y tres reglas compuestas/contextuales.
- Puedo explicar los CHECK de estados, intervalos y montos.
- Puedo distinguir una regla declarativa de una regla transaccional.
- Puedo explicar por qué el orden de creación no es arbitrario.
- Puedo explicar qué índices son de rendimiento y qué constraints protegen integridad.
- Puedo responder qué ocurriría si se elimina cualquiera de las principales protecciones.

Cierre

El objetivo no es “tener 16 tablas”. El objetivo es que cada estudiante entienda cómo el modelo de negocio se convierte en reglas que PostgreSQL puede hacer cumplir. Cuando eso está claro, SQL deja de ser una lista de comandos y se convierte en diseño de software.